Cahier des Charges - Village under attack (C++)

Présentation du Projet

Village under attack est un jeu de stratégie en temps réel dans lequel le joueur gère un village, collecte des ressources et se défend contre des ennemis. Ce projet sera développé en **C++** avec une **architecture modulaire**, en respectant les principes de la **programmation orientée objet**.

Objectifs

- Concevoir un **système économique** basé sur la gestion de l'or et de l'élixir.
- Permettre la **construction de bâtiments** et l'**entraînement de troupes**.
- Gérer **l'attaque des ennemis** et la **défense du village**.
- Implémenter un **moteur de jeu évolutif**, incluant la gestion des attaques, des ressources et des améliorations.

Architecture du Projet

1. Structure des dossiers

```
├── Buildings/
│   ├── Barrack.cpp/.h
│   ├── Building.cpp/.h
│   ├── ElixirCollector.cpp/.h
│   ├── GoldMine.cpp/.h
│   ├── ResourceGenerator.cpp/.h
│   ├── TownHall.cpp/.h
│   ├── Wall.cpp/.h
├── Entities/
│   ├── Enemies/
│   │   ├── Bomberman.cpp/.h
│   │   ├── Enemy.cpp/.h
│   │   ├── Raider.cpp/.h
│   ├── Troops/
│   │   ├── Archer.cpp/.h
│   │   ├── Barbarian.cpp/.h
│   │   ├── Troop.cpp/.h
│   ├── Entity.cpp/.h
│   ├── Npc.cpp/.h
│   ├── Player.cpp/.h
├── Board.cpp/.h
├── Position.cpp/.h
├── Resources.cpp/.h
├── main.cpp
```

2. Contraintes Techniques

- **Langage** : Utilisation exclusive de **C++ (C++17 ou supérieur)**.
- **Architecture** : Respect du paradigme **orienté objet**, avec **héritage** et **polymorphisme**.
- **Graphisme** :
  - **Première étape** : Développement d'une application en **mode console**.
  - **Deuxième étape** : Intégration d'une **interface graphique** pour améliorer l'expérience utilisateur.
  - **Technologie utilisée** : Implémentation de l'interface graphique avec la bibliothèque **SDL** pour gérer l'affichage, les animations et les interactions utilisateur.

Fonctionnalités

3. Système de Ressources

- Gestion des ressources (**or** et **élixir**) via la classe Resources.
- Ajout et consommation des ressources.
- Vérification des conditions pour acheter des éléments.

4. Bâtiments

- **Building** : Classe de base pour tous les bâtiments.
- **TownHall** (🏛) : Centre du village (**s'il est détruit, la partie est perdue**).
- **Wall** (🧱) : Protège le village en ralentissant les ennemis.
- **ResourceGenerator** : Classe de base pour les bâtiments générant des ressources.
  - **GoldMine** (⛏) : Produit de l'or.
  - **ElixirCollector** (⚗) : Produit de l'élixir.
- **Barrack** : Permet d'entraîner des troupes.

5. Entités

- **Entity** : Classe de base pour toutes les entités.
- **Npc** : Classe représentant les personnages non-joueurs.
- **Player** (👷) : Gère les interactions avec le jeu (construction, récolte, recrutement).

6. Système d'ennemis et troupes

- **Enemy** : Classe de base pour les ennemis.
  - **Raider** (🏴‍☠️) : Attaque les bâtiments.
  - **Bomberman** (💣) : Détruit les murs (ajout dans une phase avancée).
- **Troop** : Classe de base pour les troupes.
  - **Archer** (🏹) : Attaque à distance.
  - **Barbarian** (🧔) : Attaque au corps à corps.

7. Système de Carte (Board)

- Gestion du **placement des bâtiments et des unités**.
- Gestion des **déplacements et des combats**.

Phases de Développement

1. Partie 1 : Mise en Place du Village

Cette première partie concerne la mise en place du village, son économie et l'apparition des premiers ennemis. Une fois cette phase terminée, le joueur pourra se déplacer sur la carte, générer et collecter des ressources, et subir les premières attaques ennemies.

Si vous souhaitez réellement remporter la partie, la section « Partie 2 » introduira des mécaniques de défense du village, permettant de repousser les envahisseurs.

Dans ce jeu, l'économie repose exclusivement sur l'or et l'élixir :

- Les bâtiments (Buildings) s'achètent avec de l'or (sauf la GoldMine).
- Les troupes (Troops) s'entraînent avec de l'élixir, une ressource aux propriétés magiques.

Chaque élément du jeu possède une Position, définie par deux coordonnées (X, Y). Cette position est utilisée par les classes qui doivent être représentées sur la carte (Board).

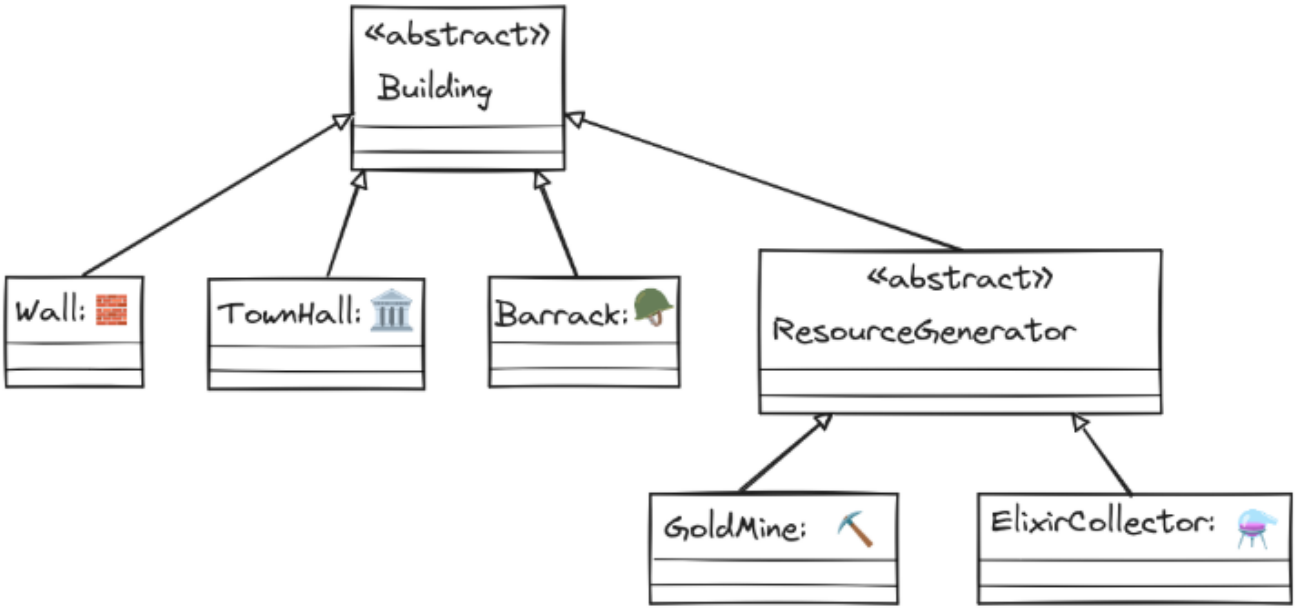
Les bâtiments (Buildings), les entités (Entities) et d'autres éléments du jeu sont entourés par un contour (Board), défini par sa largeur (SizeX) et sa hauteur (SizeY).

1.1. Construction et Bâtiments

Type	**Description**	**Coût**
TownHall 🏛️	Centre du village (**s'il est détruit, partie perdue**).	Or
GoldMine ⛏️	Produit de l'or au fil du temps.	Aucun
ElixirCollector 💧	Produit de l'élixir au fil du temps.	Or
Wall 🧱	Défend le village en ralentissant les ennemis.	Or

- Building : Classe de base pour tous les bâtiments.
- TownHall : Centre du village.
- Wall : Défense du village.
- ResourceGenerator : Classe de base pour les bâtiments produisant des ressources.
- GoldMine : Génère de l'or.
- ElixirCollector : Génère de l'élixir.
- Barrack : Permet d'entraîner des troupes.

Ci-dessous, le diagramme de classes des bâtiments, illustrant leur structure et leur relation avec le jeu.

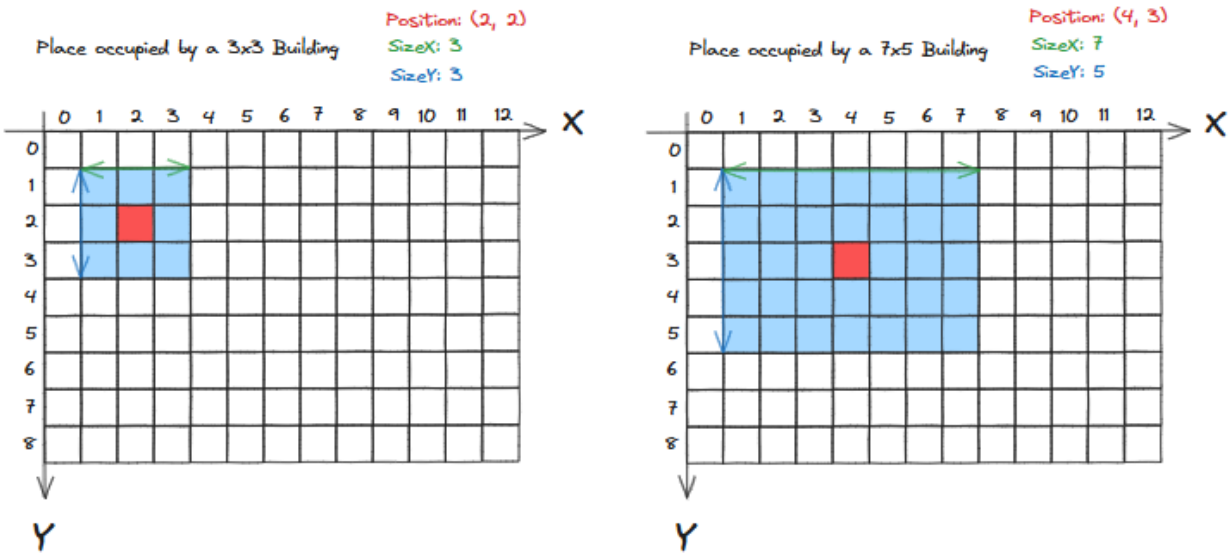


Dans cette section, **l'implémentation de la Barrack n'est pas requise**.

Un Building est défini par les propriétés suivantes :

- **SizeX** : Taille du bâtiment sur l'axe X.
- **SizeY** : Taille du bâtiment sur l'axe Y.
- **Repr** : Représentation du bâtiment sur la carte (sous forme d'emoji).
- **Cost** : Coût du bâtiment en ressources (or ou élixir).
- **MaxInstances** : Nombre maximal d'instances autorisées pour ce type de bâtiment.
- **Position** : Emplacement du bâtiment sur la carte, défini par ses coordonnées (X, Y).
- **Health** : Points de vie du bâtiment.

Les schémas suivants illustrent ces propriétés pour une meilleure compréhension des Buildings.

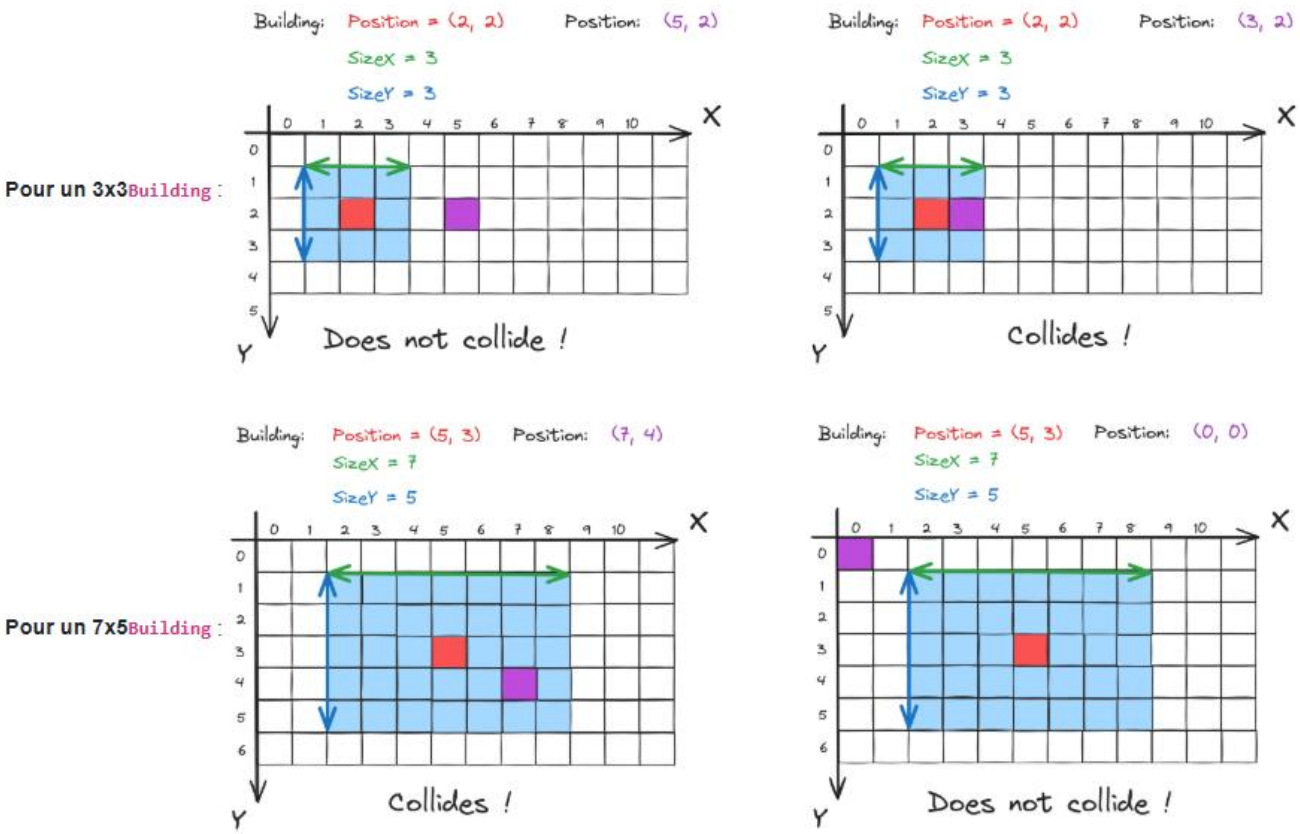


Chaque Building possède un **centre**, c'est pourquoi **SizeX et SizeY sont toujours impairs**.

Pour **collecter des ressources**, le Player doit **se rendre sur un Building** et interagir avec lui. Par exemple, pour **extraire de l'or**, le Player doit être **présent sur une GoldMine**.

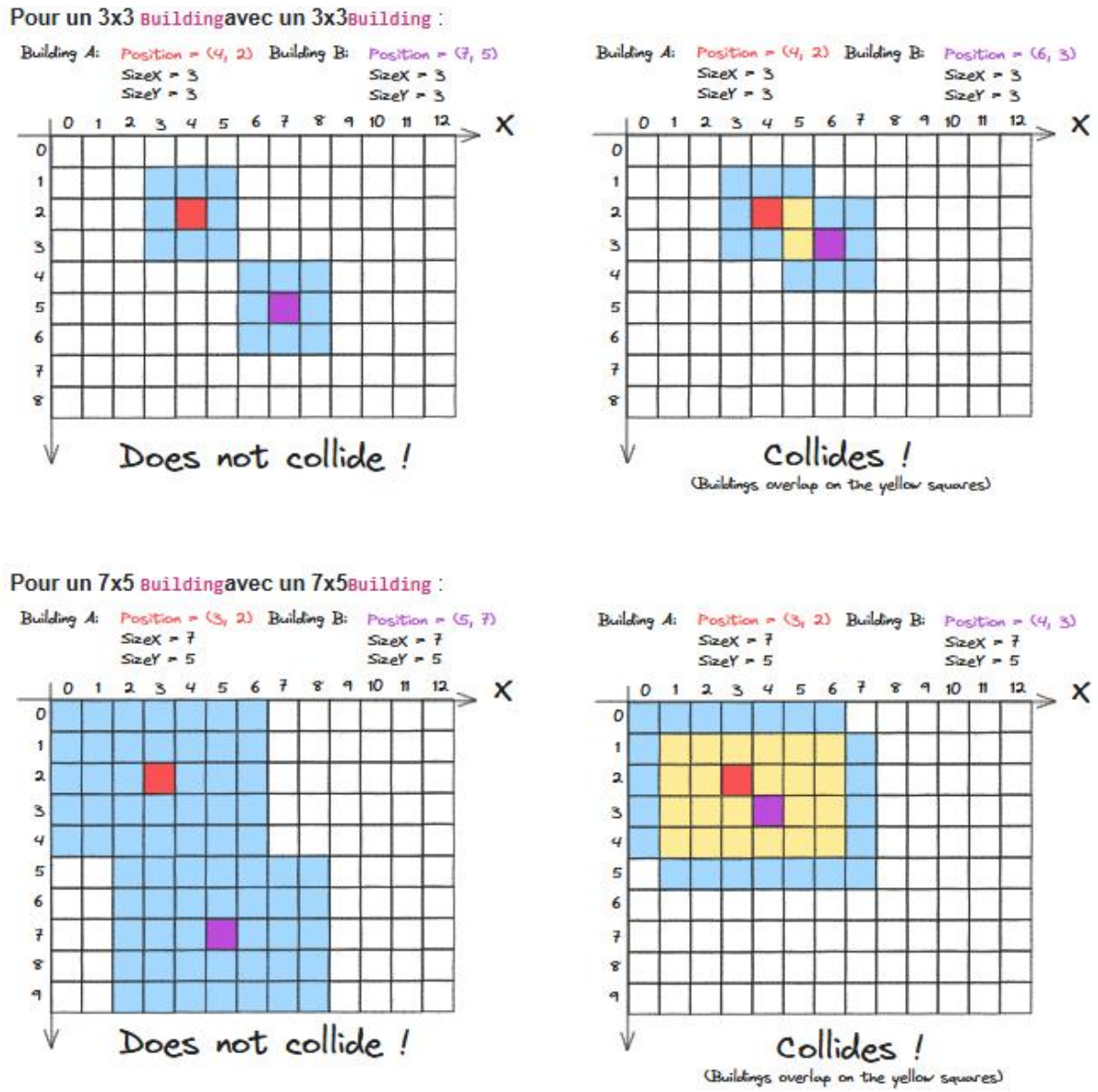
Pour déterminer si le Player se trouve sur un Building, il faut vérifier si **sa Position est incluse dans l'espace occupé par ce Building**.

Les **quatre exemples suivants** illustrent les cas où un Building **entre en collision** avec une Position.



Autrement dit, il est nécessaire de vérifier si un Building **entre en collision avec une Position**.

Pour mieux comprendre ce concept, les exemples suivants illustrent les cas où un Building **entre en collision avec un autre Building**.



Un TownHall est un Building avec les caractéristiques suivantes :

- **SizeX** = 5
- **SizeY** = 5
- **Repr** : doit être l'emoji suivant → 🏛 (copier-coller)
- **Cost** : 0 or et 0 élixir
- **MaxInstances** = 1

Un Wall est un Building avec :

- **SizeX** = 1
- **SizeY** = 1
- **Repr** : doit être l'emoji suivant → 🧱 (copier-coller)
- **Cost** : 10 pièces d'or et 0 élixir
- **MaxInstances** = 200

💡 **Amélioration des bâtiments** : Les bâtiments peuvent être améliorés pour **produire plus de ressources** ou **renforcer la défense**.

1.2. Économie et Ressources

- L'économie repose sur **deux ressources principales** :
  - 🥇 **Or** : utilisé pour construire et améliorer les bâtiments (sauf les mines d'or).
  - 🔮 **Élixir** : utilisé pour former des troupes.
- L'or est produit par les **mines d'or** (GoldMine).
- L'élixir est généré par les **collecteurs d'élixir** (ElixirCollector).
- Les ressources peuvent être **stockées** et doivent être gérées stratégiquement pour optimiser la croissance du village.

1.3. Production des Ressources

Le village doit être capable de **produire des ressources**. Pour cela, nous devons implémenter une classe ResourceGenerator, qui est un Building générant **de l'or et de l'élixir**.

La classe ResourceGenerator est définie par :

- **Current** : quantité actuelle de ressources contenues dans le générateur.
- **Capacity** : quantité maximale que le générateur peut contenir (fixée à **100**).
- **IsFull** : booléen indiquant si le générateur a atteint sa capacité maximale.
  Un ResourceGenerator est **plein** lorsque Current est **supérieur ou égal à** Capacity.

1.4. Exploitation des Mines

GoldMine (⛏️ ou ▯)

Une GoldMine est un ResourceGenerator avec les caractéristiques suivantes :

- **SizeX** = 3
- **SizeY** = 3
- **Cost** : 0 or et **100 élixirs**
- **MaxInstances** = 3
- **Repr** :
  - Si pleine → ▯ (copier-coller)
  - Sinon → ⛏️ (copier-coller)

ElixirCollector (🧪 ou 🔮)

Une ElixirCollector est un ResourceGenerator avec :

- **SizeX** = 3
- **SizeY** = 3
- **Cost** : **100 pièces d'or** et 0 élixir
- **MaxInstances** = 3

- **Repr** :
  - Si plein → 🔮 (copier-coller)
  - Sinon → 🔭 (copier-coller)

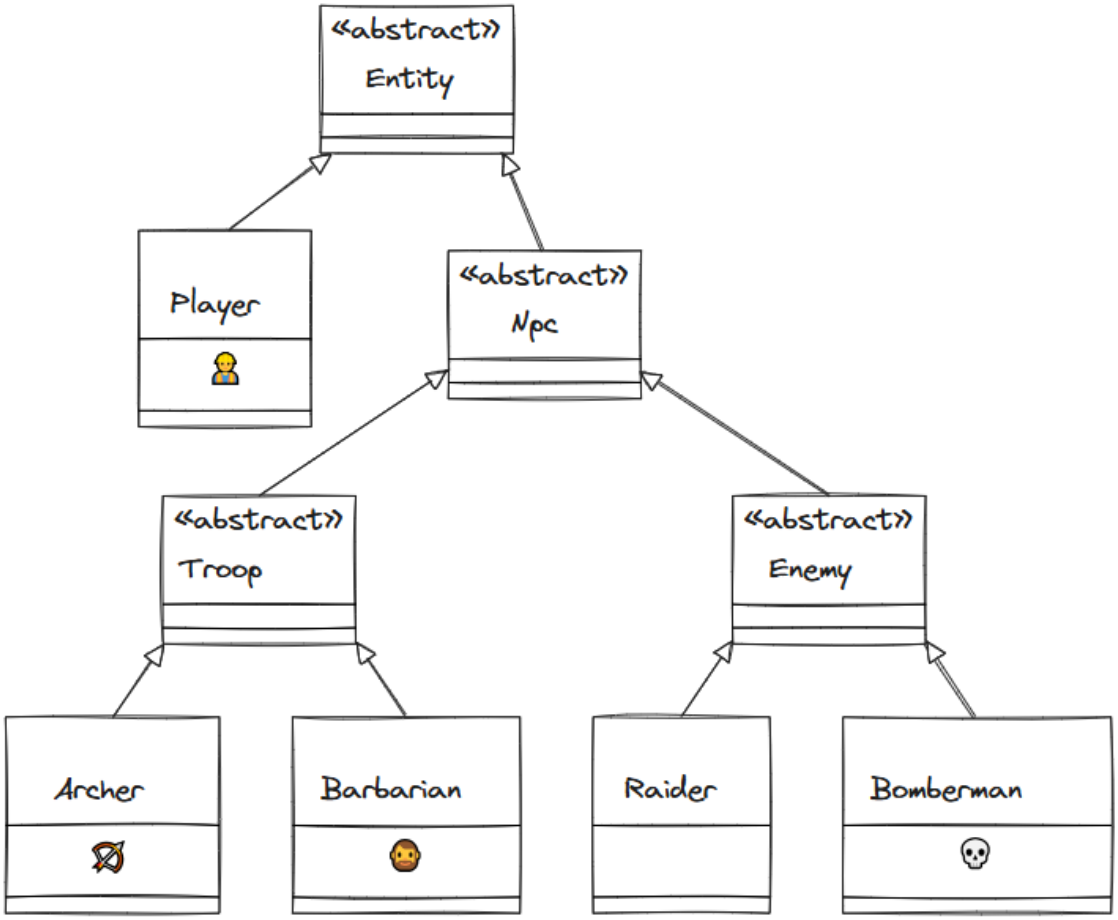
1.5. Entités du Village

Le village est peuplé de nombreuses créatures vivantes appelées **Entities**. Parmi elles, on retrouve :

- **Le Player** : personnage contrôlé par le joueur.
- **Les Troops** : troupes utilisées pour attaquer ou défendre.
- **Les Enemies** : ennemis qui attaquent le village.

💡 **Système de combat** :

- Les troupes du joueur **défendent** le village contre les ennemis.
- Les ennemis **attaquent automatiquement** en visant le TownHall.
- Les **bâtiments n'ont pas d'attaque** dans cette version.
- **Si le TownHall est détruit, la partie est perdue**.



Type	Rôle	Coût (en élixir)
Archer 🏹	Attaque à distance.	Moyen
Barbare 🧔	Attaque au corps à corps.	Faible
Raider 🏴‍☠️	(Ennemi) Attaque les bâtiments.	-
Bomberman 💣	(Ennemi) Détruit les murs.	-

Information

*L'implémentation des **Troop**, **Archer**, **Barbarian** et **Bomberman** sera réalisée dans la **section 2 « Amélioration et Défense»**.*

1.5.1 Définition d'une Entity

Une Entity est caractérisée par :

- **Position** : la position de l'Entity sur la carte (coordonnées X, Y).
- **Repr** : la représentation graphique de l'Entity (sous forme d'emoji ou de caractère).

1.5.2. Player : le personnage du joueur

Le **Player** a pour rôle de **défendre son village** en :

- Construisant des **Buildings**.
- Collectant des **Resources**.
- Entraînant des **Troops**.

La classe Player remplace une propriété de sa classe parente :

- **Repr** : doit être représenté par l'emoji → 👷 (copier-coller).

De plus, le Player possède :

- **Resources** : les ressources (or et élixir) actuellement en sa possession.

a. Npc : entités non-joueurs

La classe **Npc** représente toutes les **entités qui ne sont pas le Player**. Elle est définie par :

- **Health** : les points de vie de l'Npc.

Partie 2 : Amélioration et Défense

- Ajout de la **caserne (Barrack)** pour entraîner des troupes.
- Ajout de **nouvelles unités** (Archer, Barbarian, Bomberman).
- Amélioration du **système de combat et des attaques ennemies**.

Partie 3 : Optimisation et Interface Graphique

- Optimisation du moteur de jeu.
- Développement d'une **interface graphique (SDL ou SFML)**.
- Tests et corrections de bugs.

Consignes

Vous pouvez déplacer le joueur à l'aide des flèches de votre clavier.
Vous pouvez construire de nouveaux bâtiments en dépensant les ressources nécessaires, grâce aux touches associées :

- Touche G -> GoldMine
- Touche E -> ElixirCollector
- Touche W -> Wall

La GoldMine et l'ElixirCollector génèrent des ressources, mais c'est à vous de les collecter : vous devez vous tenir dessus et appuyer sur la touche C.

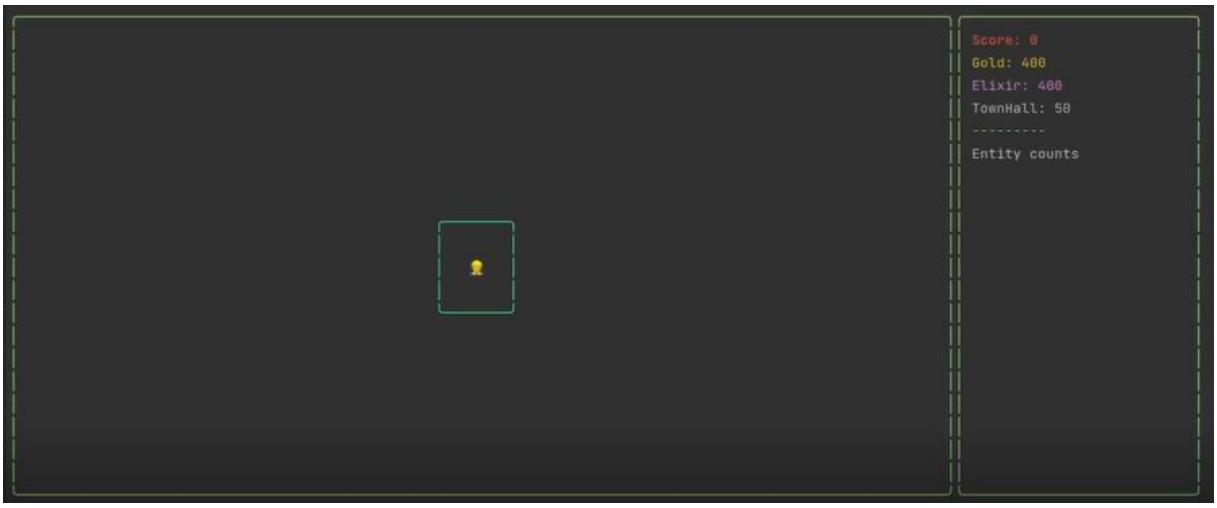
Les Raiders commenceront à apparaître après un délai fixe et se concentreront sur le bâtiment le plus proche. Vous pouvez commencer à les ralentir en construisant des Walls, mais s'ils parviennent à détruire votre TownHall, ce sera la fin.

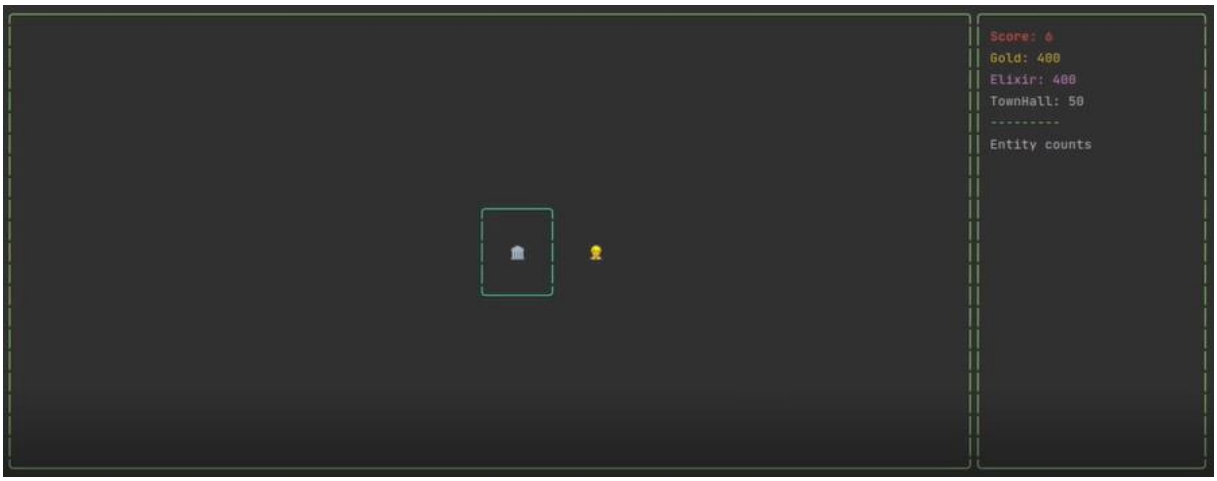
Les images suivantes représentent l'interface de jeu en mode **console.** L'écran est divisé en **deux parties** :

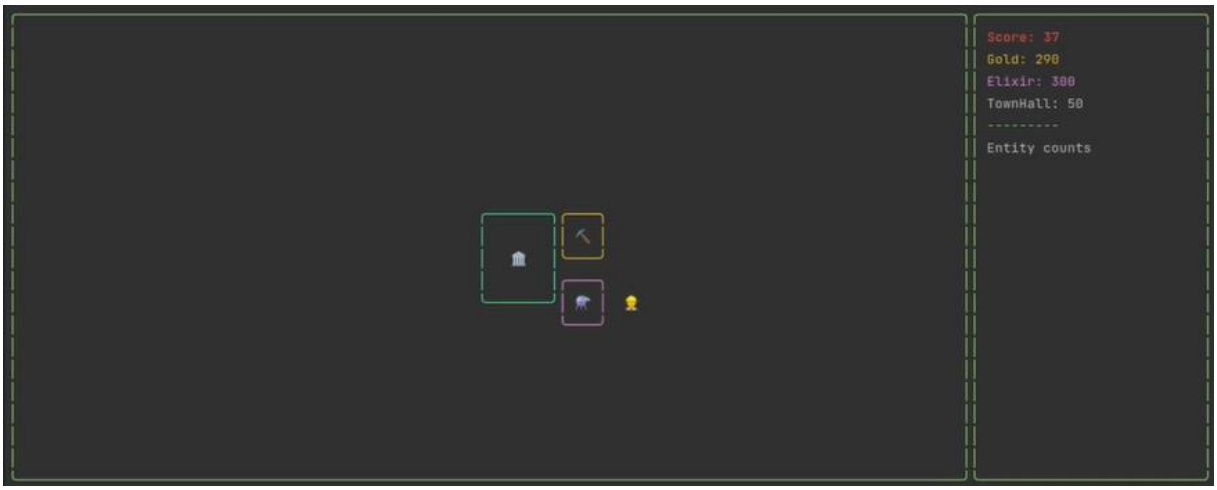
- **Zone principale (gauche)** : Une grande aire de jeu où l'on voit le joueur gère des bâtiments et des ressources.
- **Panneau d'informations (droite)** : Affichage des ressources et des statistiques du jeu :
  - **Score** :
  - **Gold** :
  - **Elixir** :
  - **TownHall** :
  - **Entity counts** :

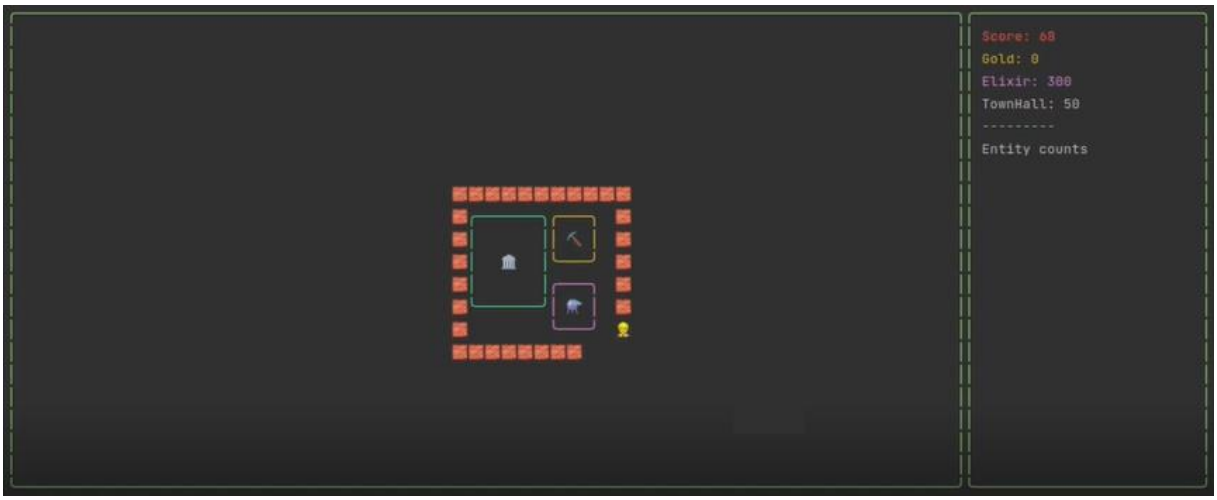
La **première image** montre l'état initial du jeu

Le joueur commence avec un certain capital (**Gold : 400** et **Elixir : 400**)

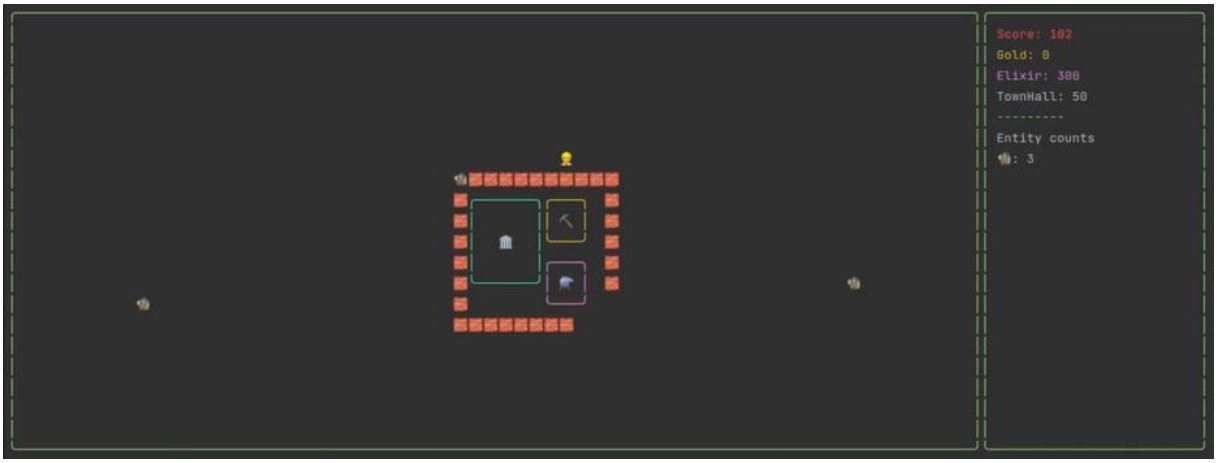
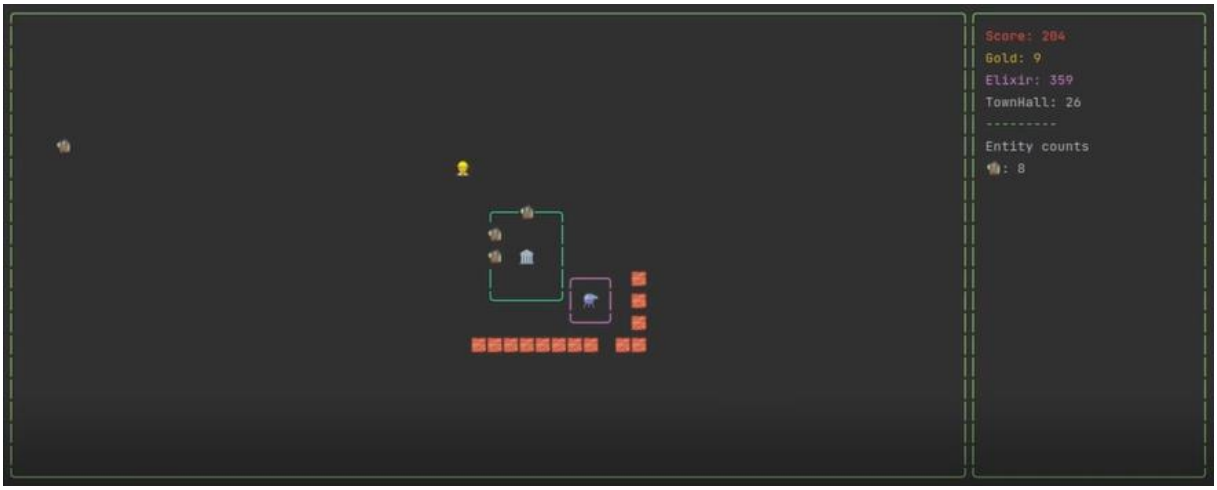
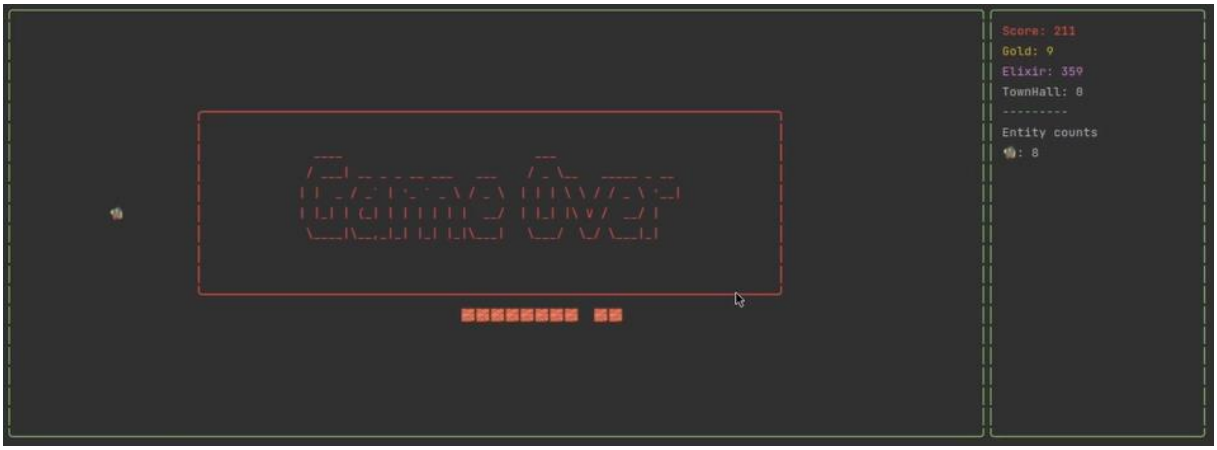








L'image ci-dessus montrent les ennemis qui commencent à attaquer le village



Une fois le TownHall (hôtel de ville) est détruit, cela signifie souvent la fin de la partie pour un joueur.